

IC526 Advanced Compilers

Lec 05: Automata Basics II

Yoonseung Kim

DGIST EECS

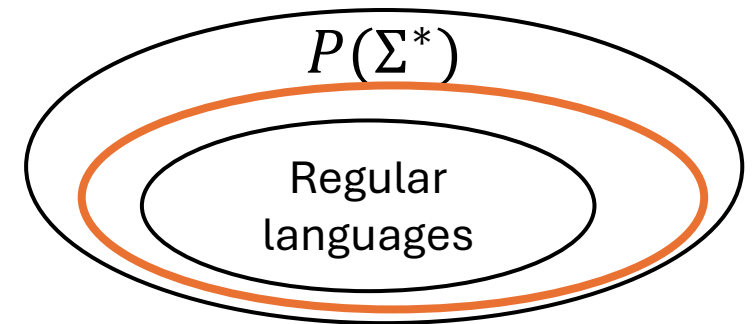
Fall 2026

Where We Are

- Lec 01: The course overview
- Lec 02: OCaml – The development language
- Lec 03: Rust – The source language
- Lec 04: Automata basics I – DFA/NFA/RE (for lexers)
- Lec 05: Automata basics II – PDA/CFG (for parsers)

Recap: DFA / NFA / RE

- DFA: a deterministic machine that recognizes a language.
- Every NFA can be converted into an equivalent DFA.
- Regular expression (RE) is a notation for describing a language.
- For a language L , these are equivalent:
 - $L = L(r)$ for some regular expression r
 - $L = L(M)$ for some DFA M
 - $L = L(N)$ for some NFA N
- Moreover, these equivalences are constructive:
 - $RE \Rightarrow NFA \Rightarrow DFA$ (and $DFA \Rightarrow NFA \Rightarrow RE$)



Today, we will study a **larger class** of languages.

Pushdown Automata (PDA)

Limitation of DFA / NFA / RE

- Consider the language of balanced parentheses.

$$\Sigma = \{ (,) \} \quad L = \{ \epsilon, (), (()), ()(), (()()), \dots \}$$

$$\underbrace{)(, ())}_{\text{Closes when nothing is open}}, \underbrace{(()())}_{\text{Not properly closed}}, \dots \notin L$$

Closes when nothing is open

Not properly closed

- Question:
Define a DFA or NFA that recognizes this language.
Or show that such a DFA is impossible.

Impossibility Reasoning

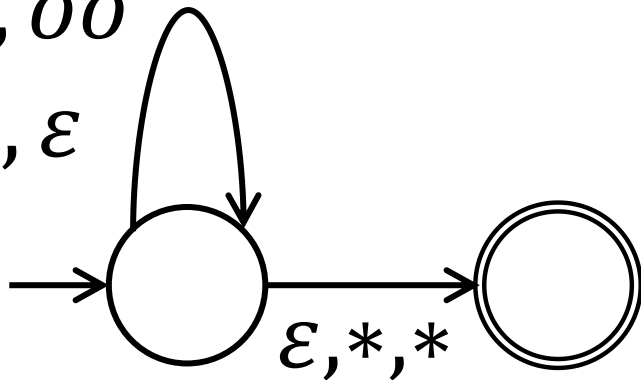
- Suppose there is such a DFA machine M .
- Let n be the number of states in M . (DFA means states are finite.)
- Now, consider these $(n+1)$ strings:
 $(, ((, (((, \dots, ((((((... (\rightarrow$ equivalently, $(^1, (^2, \dots, (^{n+1}$
- Then, there must be two strings $(^k$ and $(^l$, where $k \neq l$, that go to the same state q from the initial state of M . (pigeonhole principle)
- Then, from q , both $)^k$ and $)^l$ go to a final state, regardless of the past.
- Contradiction: $(^k)^l$ is accepted by M , which means M does not recognize the language L .

Pushdown Automata (PDA)

- A PDA is a (nondet.) finite automaton equipped with a stack.

$(, *, o *$ Label: (input_symb, stktop_to_pop, str_to_push)

$(, o, oo$
 $), o, \varepsilon$



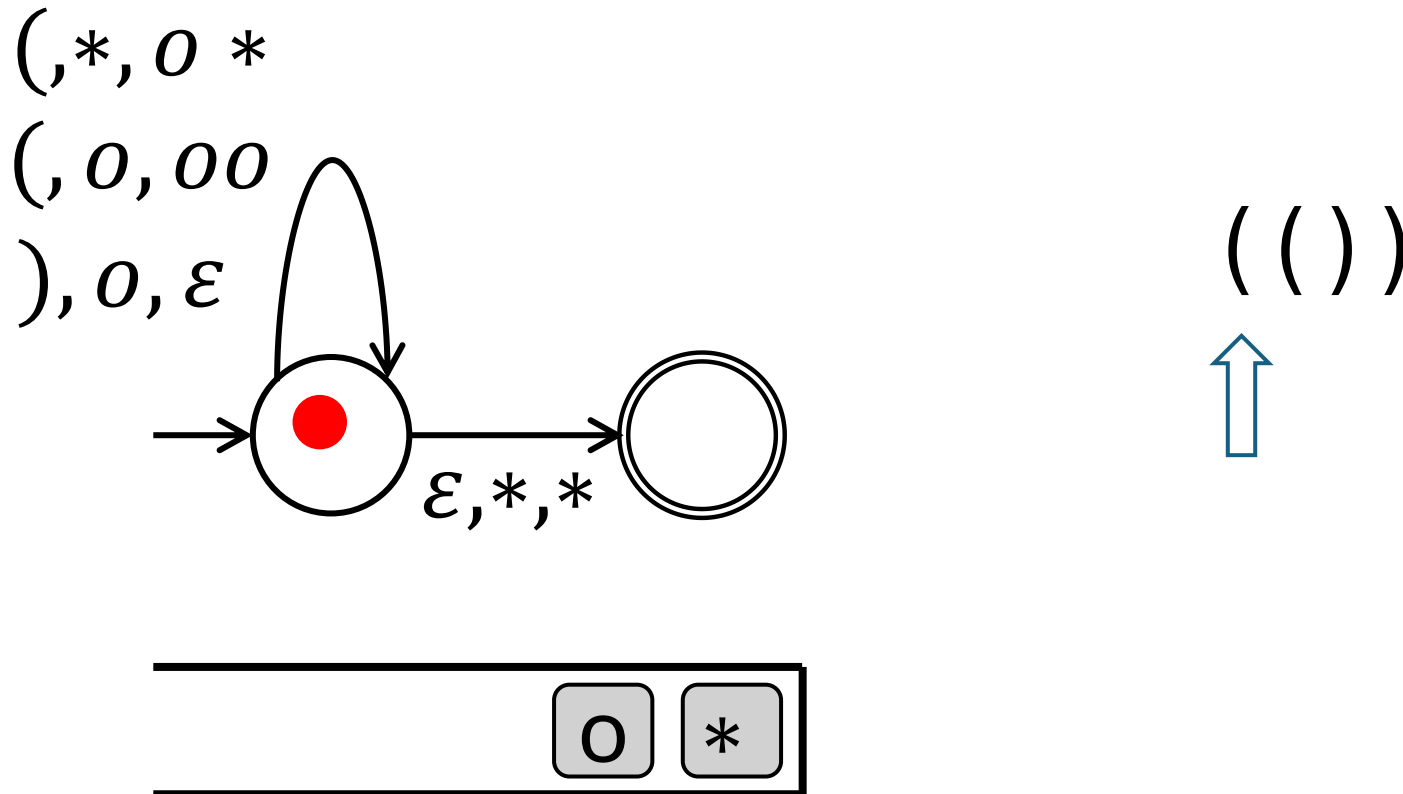
Input alphabet $\Sigma = \{ (,) \}$



Stack alphabet $\Gamma = \{ *, o \}$

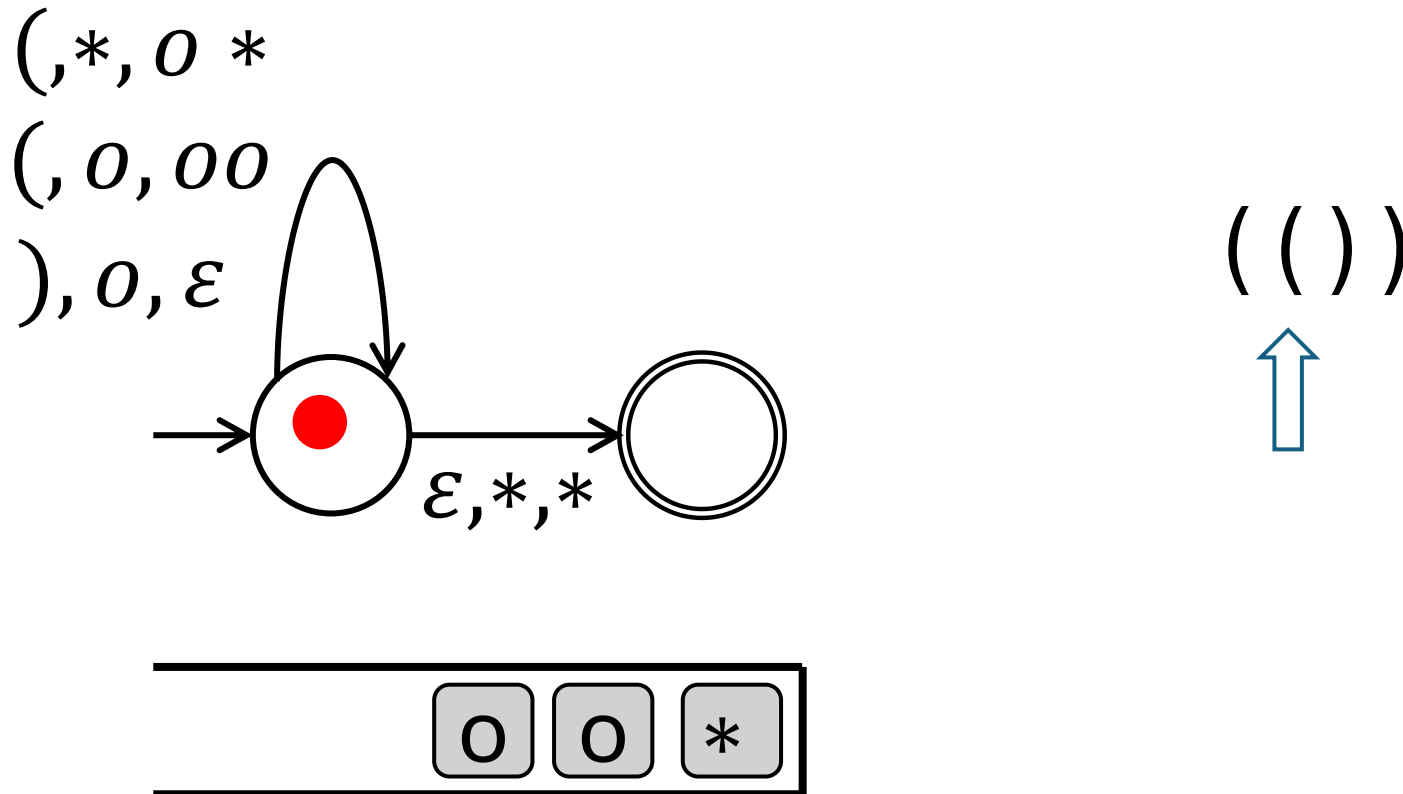
Pushdown Automata (PDA)

- A PDA is a (nondet.) finite automaton equipped with a stack.



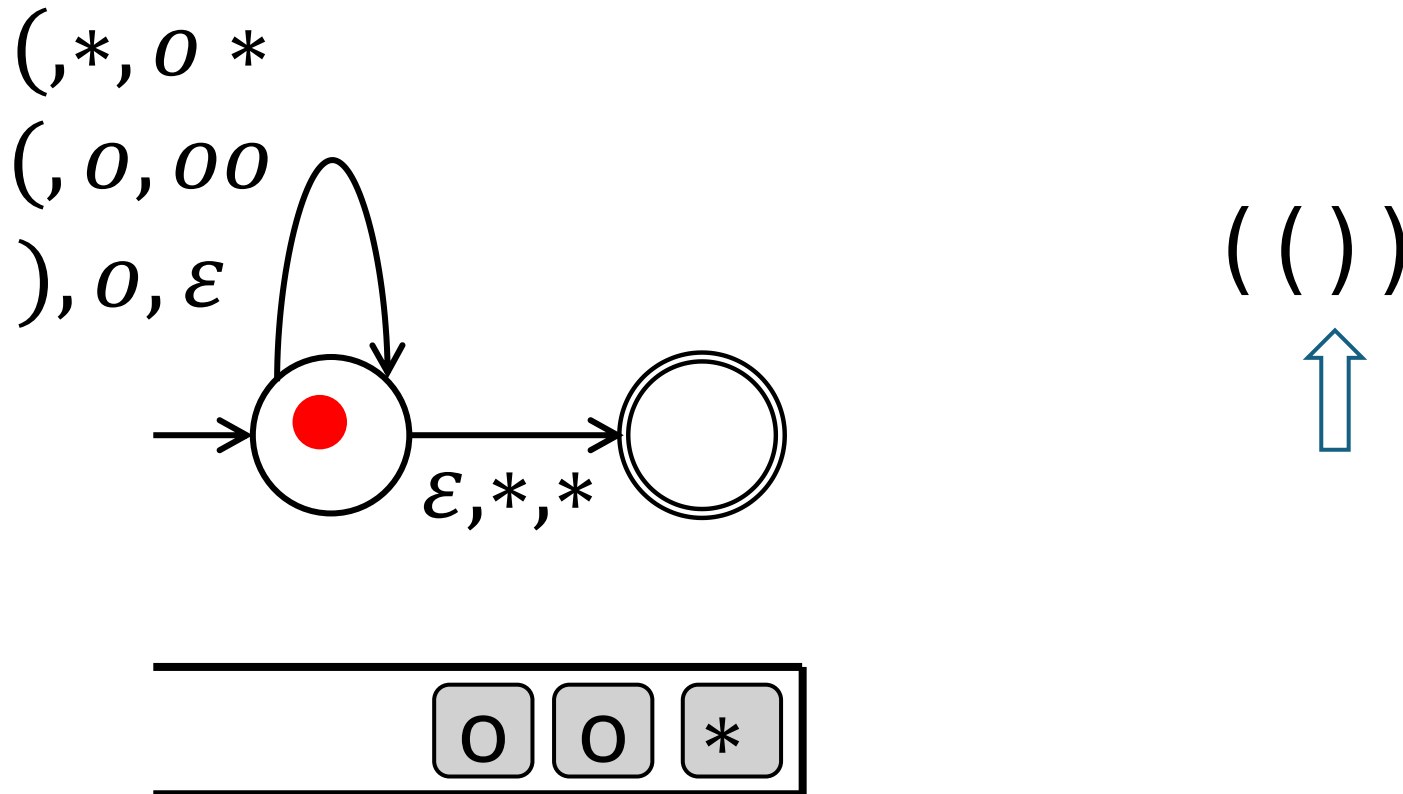
Pushdown Automata (PDA)

- A PDA is a (nondet.) finite automaton equipped with a stack.



Pushdown Automata (PDA)

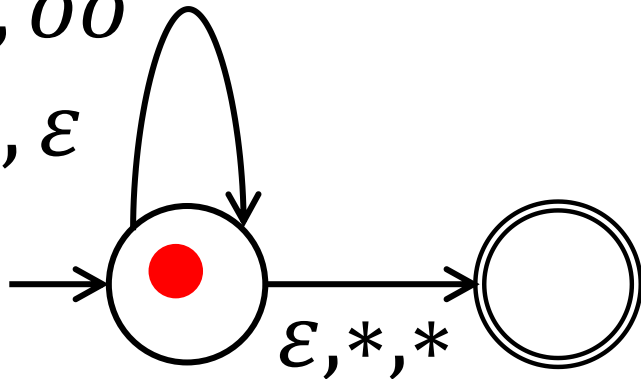
- A PDA is a (nondet.) finite automaton equipped with a stack.



Pushdown Automata (PDA)

- A PDA is a (nondet.) finite automaton equipped with a stack.

$(, *, 0 *$
 $(, 0, 00$
 $), 0, \varepsilon$



$(())$
↑



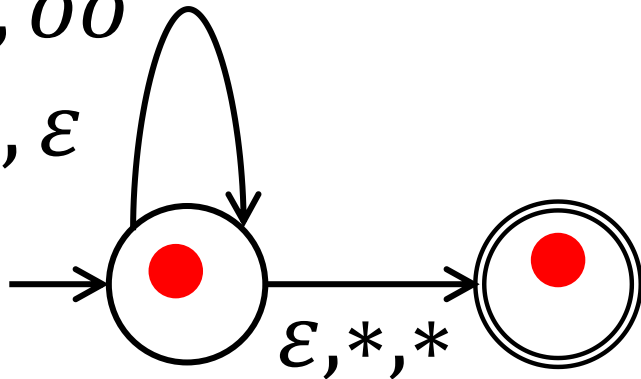
Pushdown Automata (PDA)

- A PDA is a (nondet.) finite automaton equipped with a stack.

$(, *, o *$

$(, o, oo$

$), o, \varepsilon$

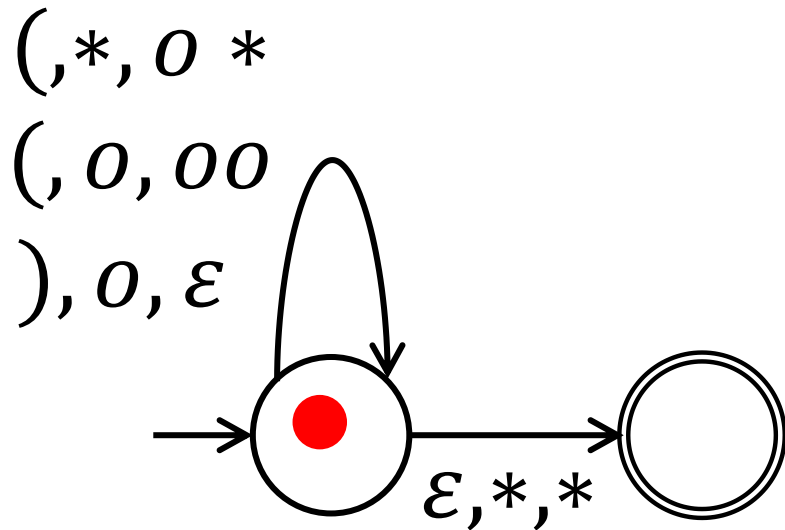


$(())$



Pushdown Automata (PDA)

- A PDA is a (nondet.) finite automaton equipped with a stack.



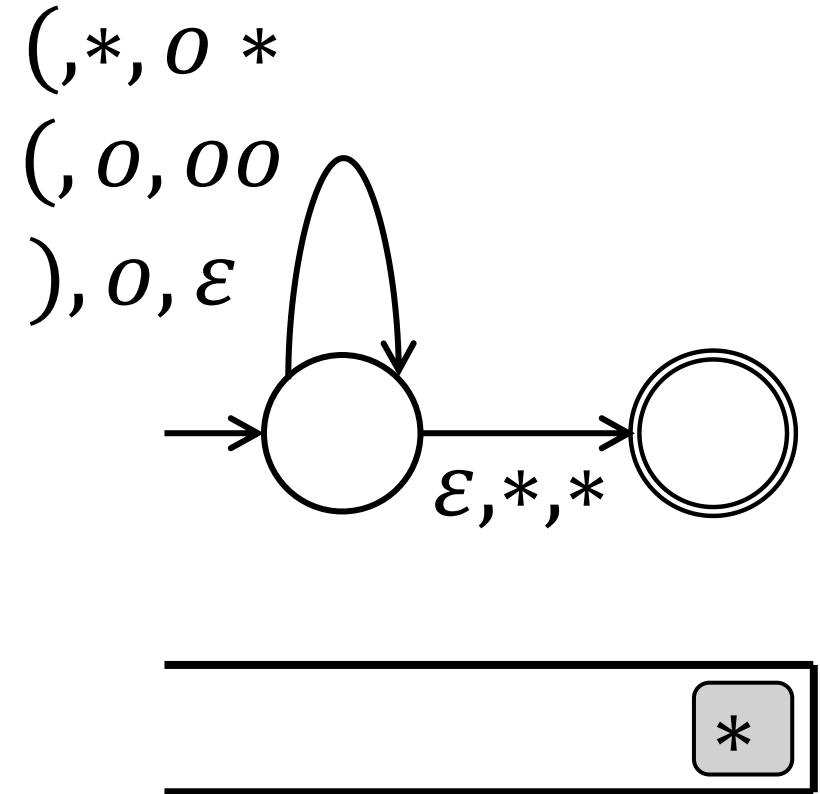
[Rejected Input]

$())$



PDA: Formal Definition

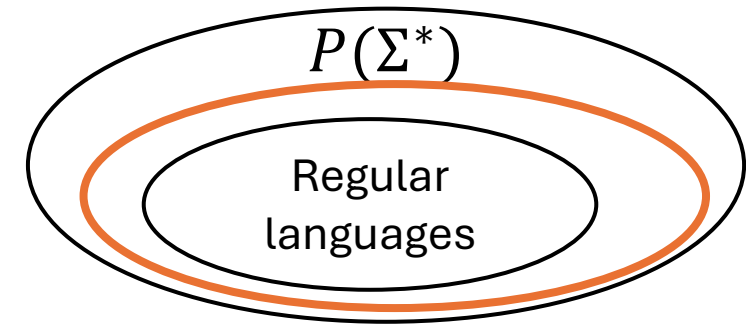
- A PDA $M = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$
 - Q (Finite set of states)
 - Σ (Finite set of input symbols)
 - Γ (Finite set of stack symbols)
 - $\delta: Q \times (\Sigma \cup \{\varepsilon\}) \times \Gamma \rightarrow \mathcal{P}_{\text{fin}}(Q \times \Gamma^*)$
(Transition function)
 - $q_0 \in Q$ (Initial state)
 - $Z_0 \in \Gamma$ (Initial stack symbol)
 - $F \subseteq Q$ (Final states)



Exercise:
Identify each component.

Where We Are & Where We're Going

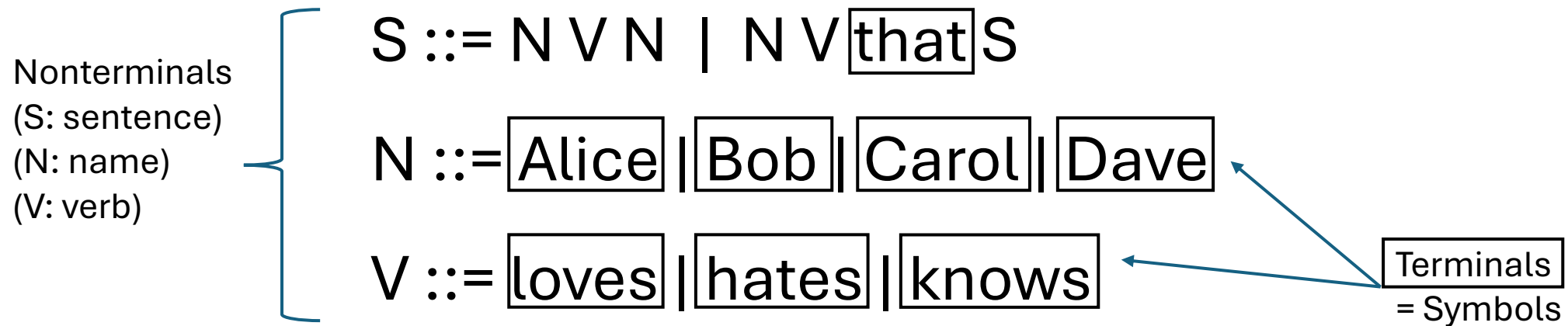
- If a language is regular:
 - DFA/NFA can recognize them.
 - RE is a representation method.
- PDA recognizes a **larger class** of languages.
 - The parentheses language
 - RustY's syntax (=grammar)
- Question: Is there a notation for representing these languages?
 - > Yes, and that is **Context-Free Grammar**.



Context-Free Grammars (CFGs)

Context-Free Grammar (CFG): Intuition

- Consider the following patterns for English sentences:



You can *derive* a sentence by:

- (1) starting from S;
- (2) repeatedly replacing a **nonterminal** using one of its **production rules**.

$S \rightarrow N V \text{that} S \rightarrow N V \text{that} N V N \rightarrow \text{Alice } V \text{that} N V N$
 $\rightarrow \dots \rightarrow \text{Alice knows that Carol loves Dave}$

Backus-Naur Form

- A standard concrete notation for writing production rules

$$S ::= N V N \mid N V \boxed{\text{that}} S$$
$$N ::= \boxed{\text{Alice}} \mid \boxed{\text{Bob}} \mid \boxed{\text{Carol}} \mid \boxed{\text{Dave}}$$
$$V ::= \boxed{\text{loves}} \mid \boxed{\text{hates}} \mid \boxed{\text{knows}}$$

- $\langle \text{nonterminal} \rangle ::= \langle \text{expr}_1 \rangle \mid \langle \text{expr}_2 \rangle \mid \dots$
 - **Nonterminals** are temporary symbols (such as S, N, V).
 - **Terminals** are the actual symbols that appear in derived strings ($\{\text{that}, \text{Alice}, \dots\}$.)
 - Each expr_k is a string of nonterminals and terminals (e.g., N V that S.)

$$w \in L(G) \Leftrightarrow w \text{ is derivable from } G$$

CFG: Formal Definition

- A CFG $G = (V, \Sigma, R, S)$
 - V (Finite set of nonterminals)
 - Σ (Finite set of terminals)
(where $V \cap \Sigma = \emptyset$)
 - $R \in \mathcal{P}_{\text{fin}}(V \times (V \cup \Sigma)^*)$
(Production rules)
 - $S \in V$ (Start symbol)

$S ::= N V N \mid N V \text{that} S$

$N ::= \text{Alice} \mid \text{Bob} \mid \text{Carol} \mid \text{Dave}$

$V ::= \text{loves} \mid \text{hates} \mid \text{knows}$

Exercise:

Identify each component.

What Context-Free Means

- A nonterminal can be replaced with any of the rules, **regardless of the context.**
- In other words, LHS of a production rule is a single nonterminal.

Alice V that S → (1) Alice loves that S
 (2) Alice hates that S

Dave V that S → (1) Dave loves that S
 (2) Dave hates that S

Because $V ::= \text{loves} \mid \text{hates} \mid \text{knows}$

CFG Example 2: The Parentheses Language

$$S ::= (S) S \mid \varepsilon$$

Derivation of $()$: $S \rightarrow (S)S \rightarrow (S) \rightarrow ((S)S) \rightarrow \dots \rightarrow ()$

Derivation of $()()$: $S \rightarrow (S)S \rightarrow (S) \rightarrow ((S)S) \rightarrow$
 $((S)S) \rightarrow ((S)S)S \rightarrow ((S)S) \rightarrow$
 $((S)S)((S)S) \rightarrow \dots \rightarrow ()()$

CFG Example 3: The Expression Language

$E ::= E + E \mid E * E \mid n$ (where $n \in \mathbb{N}$)

Derivation of $3 + 2 * 5$:

E

$\rightarrow E + E$

$\rightarrow E + E * E$

$\rightarrow E + 2 * E$

$\rightarrow E + 2 * 5$

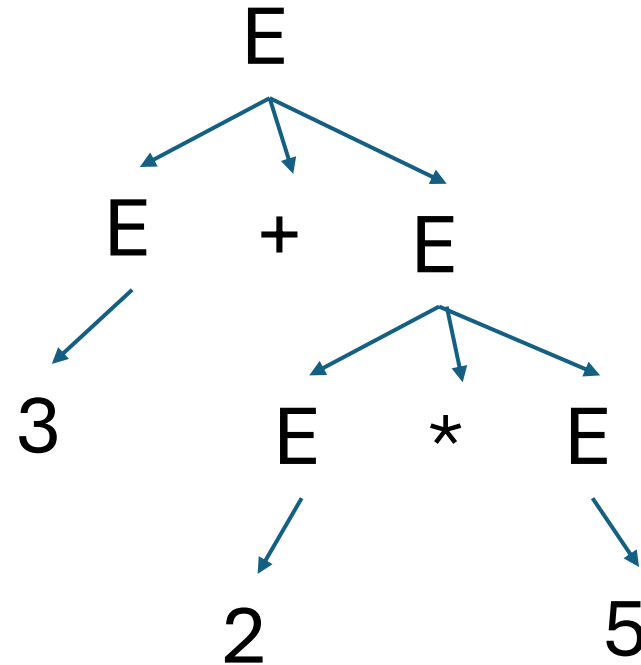
$\rightarrow 3 + 2 * 5$

- 
- We still have finite rules, as long as we don't distinguish numbers.
 - You will get a concrete understanding after learning the concept of *tokens*.

Derivation Tree Reveals Structure

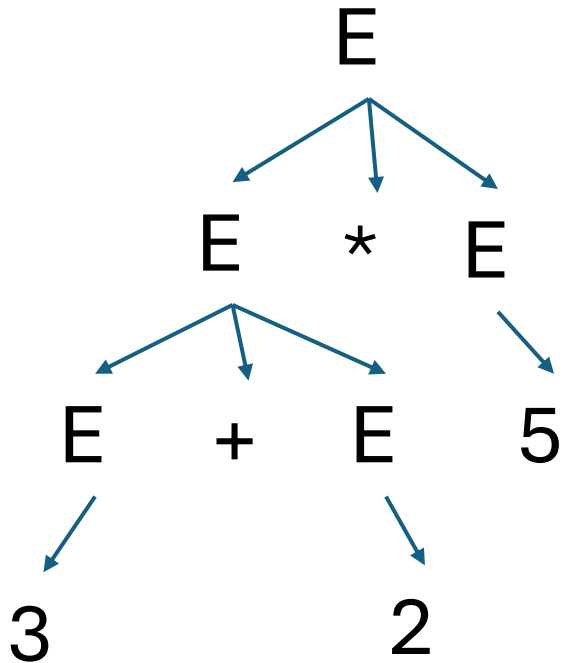
$E ::= E + E \mid E * E \mid n \quad (\text{where } n \in \mathbb{N})$

E
 $\rightarrow E + E$
 $\rightarrow E + E * E$
 $\rightarrow E + 2 * E$
 $\rightarrow E + 2 * 5$
 $\rightarrow 3 + 2 * 5$



Derivation Orders

$E ::= E + E \mid E * E \mid n \quad (\text{where } n \in \mathbb{N})$



The order does not affect the result, but fixing it gives each tree a single **canonical derivation**.

Leftmost Derivation

E
 $\rightarrow E * E$
 $\rightarrow E + E * E$
 $\rightarrow 3 + E * E$
 $\rightarrow 3 + 2 * E$
 $\rightarrow 3 + 2 * 5$

Rightmost Derivation

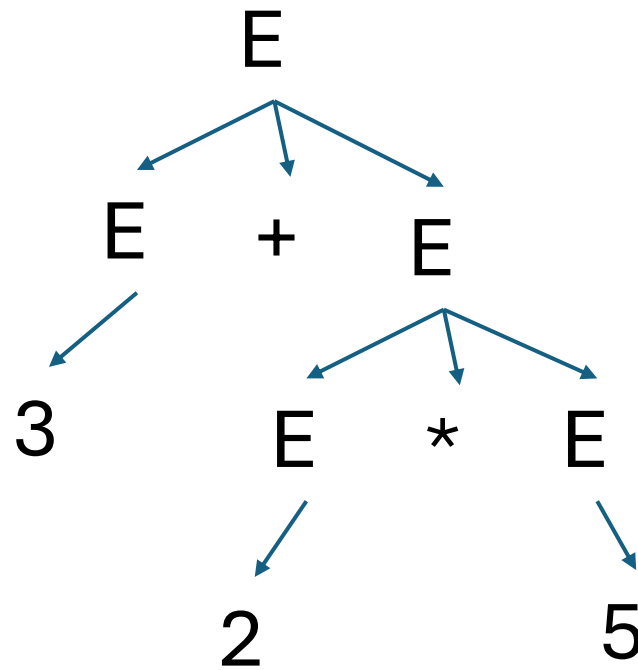
E
 $\rightarrow E * E$
 $\rightarrow E * 5$
 $\rightarrow E + E * 5$
 $\rightarrow E + 2 * 5$
 $\rightarrow 3 + 2 * 5$

Ambiguity – When Two derivation Trees Exist

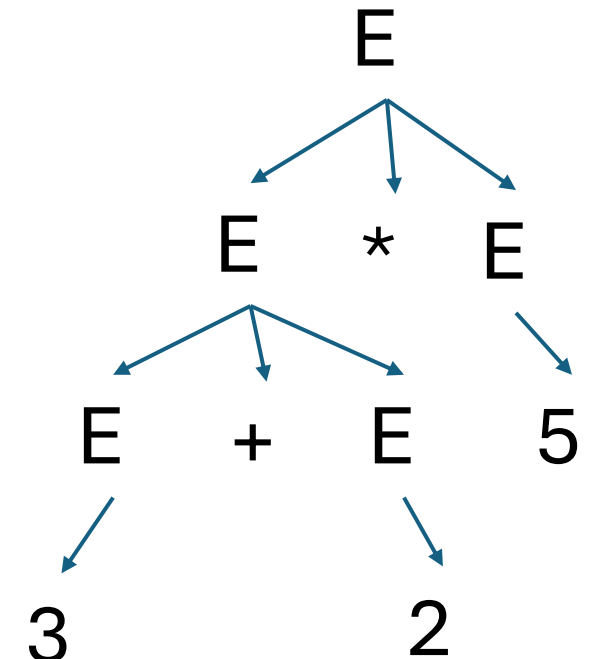
A grammar is **ambiguous** if a string may have two derivation trees.

$E ::= E + E \mid E * E \mid n$ (where $n \in \mathbb{N}$)

E
 $\rightarrow E + E$
 $\rightarrow E + E * E$
 $\rightarrow E + 2 * E$
 $\rightarrow E + 2 * 5$
 $\rightarrow 3 + 2 * 5$



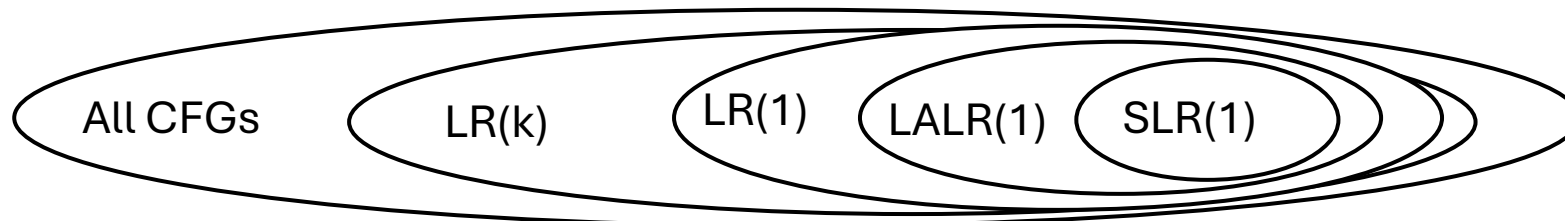
E
 $\rightarrow E * E$
 $\rightarrow E + E * E$
 $\rightarrow E + 2 * E$
 $\rightarrow E + 2 * 5$
 $\rightarrow 3 + 2 * 5$



Q: Which one is the **'unwanted'** tree?

From CFGs to Practical Parsers

- $\text{CFG} \Leftrightarrow \text{PDA}$ (In this course, let's just accept it!)
 - Every CFG has an equivalent PDA.
 - However, the resulting PDA may be nondeterministic .
- Unsurprisingly, $\text{PDA} \not\Rightarrow \text{Deterministic PDA}$
 - Recall: $\text{NFA} \Rightarrow \text{DFA}$, due to the finiteness of states
 - $\text{PDA} \not\Rightarrow \text{DPDA}$ due to the infinity of the combination (state x stack)
- For some grammars, we can still construct DPDAs (= parsers)



E.g., LR(1) grammars can be recognized by LR(1) parsers.

The **KEY** Takeaway

- Some languages can be represented as RE, and in that case, we can construct DFA that recognize them.
- Some languages are more complex so that they need CFG to represent them, and in that case, PDA recognizes them.
- However, PDA in general are nondeterministic, so only a part of CFG can be recognized by DPDA.